

Member 1: Data Pipeline & FortyGuard API Integration (David)

Core Focus: Data ingestion, API client reliability, and caching infrastructure.

- **FortyGuard Client Wrapper:**

- Implement the Python client wrapping `POST /v1/env_params`, `POST /v1/heatmap`, and `POST /v1/heat_intelligence`.
- Implement the asynchronous submit-and-poll helper for `GET /v1/status/{activity_id}` with exponential backoff handling.
- Track quota usage via `POST /v1/system/fetch-api-key-usage`.

- **Site Configuration & Data Standardization:**

- Build the metadata registry for 2–3 target U.S. data center locations (e.g., Ashburn VA, Phoenix AZ, San José CA) with coordinates, rated IT load, and baseline cooling specs.
- Normalize incoming responses into structured Pandas DataFrames covering the 12-hour hourly forecast and environmental metrics (dry-bulb temperature, relative humidity, AQI, solar irradiance).

- **Offline Caching & Reliability:**

- Build a local caching layer (SQLite or pre-serialized JSON files) to store fetched API payloads.
 - Create mock data generators for local offline development and zero-latency demo presentation.
-

Member 2: Decision Engine, ML Modeling & Orchestration (Ahmed)

Core Focus: Psychrometric cooling logic, efficiency impact calculations, and LLM narrative pipeline.

- **ASHRAE Cooling Recommendation Engine:**

- Implement psychrometric threshold rules to classify conditions into **Free-Air**, **Evaporative**, or **Mechanical DX Cooling**.
- Incorporate air quality (AQI) thresholds to prevent evaporative cooling during high-particulate periods.

- **Energy, PUE & Cost Modeling:**

- Build the PUE / WUE calculation engine comparing the recommended mode against an always-mechanical baseline.
- Add the financial translation module: convert estimated compressor energy savings into hourly cost deltas (*Dollar/hour*) and avoided carbon emissions (*kg CO₂e*).

- **Pipeline Orchestration & Narrative Layer:**

- Set up the LangChain/LangGraph workflow to sequence: Fetch Data → Run Decision Engine → Generate Alert Triggers → Format Output Schema.
- Build the GPT-4o narrative prompt integration (via GitHub Models or OpenAI client) to translate raw metrics into operational risk notes and alert summaries.

Member 3: Frontend Dashboard & Visual Analytics (Ramy)

Core Focus: Streamlit application architecture, geospatial mapping, and visual UX.

- **Streamlit Application Layout:**
 - Implement the dashboard UI layout adhering to the FortyGuard light/sky-blue visual aesthetic.
 - Build the sidebar facility switcher displaying facility status, IT load, and active cooling state.
- **Time-Series Charts & KPI Displays:**
 - Build the interactive 12-hour forecast timeline showing ambient temperature trends and cooling mode switch points.
 - Construct the KPI metric cards for live ambient temperature, recommended mode, estimated PUE delta, and cost savings.
 - Build the Heat Risk Narrative and Alert panel to render dynamic text from the LLM layer.
- **Geospatial & Heatmap Visualizations:**
 - Render GeoJSON polygon overlays or raster heatmaps for the data center facilities using Folium or PyDeck.
 - Add the multi-facility comparison table view (DATS-style) for side-by-side screening.

Task Matrix & Integration Milestones

Timeline	David (Data / API)	Ahmed (Logic / LLM)	Ramy (UI / Frontend)
Aug 24–25 (Sprint 1)	API client, polling logic, caching layer, and facility dataset schema.	ASHRAE cooling-mode rules and PUE/cost calculation engine.	Streamlit skeleton layout, facility selector, and dummy KPI metric cards.
Aug 26–27 (Sprint 2)	Heatmap and Heat Intelligence payload	LangChain/LangGraph pipeline and GPT-4o	12-hour interactive timeline chart and

Timeline	David (Data / API)	Ahmed (Logic / LLM)	Ramy (UI / Frontend)
	parser.	narrative generation.	Folium/PyDeck map integration.
Aug 28–29 (Sprint 3)	API credit monitor integration and offline demo dataset preparation.	End-to-end output verification and edge-case error handling.	Multi-facility comparison view and UI theme polish.
Aug 30 (Submission)	Demo recording support and GitHub repository documentation / README setup.	Written summary draft (Problem → User → API → Measured Result).	Final end-to-end testing, UI screenshots, and deployment verification.